



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2025-2026

Section: K

Project Report On

Project Name : Property Rental Service

Supervised By

Dr. Md. Iftekharul Mobin

Submitted By:

Name				ID	
1. A. M. Nurunnabi Azad				24-57396-2	
2. Mohammad Tahsinur Rahman				24-57209-2	
3. Raghieb Raonak Islam				24-57722-2	
4.Anika Bushra Priyanka				24-57723-2	
Obtained Marks for CO2 and CO3 (Description given in the following page)					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria (CO2)	Total =		Evaluation Criteria (CO3)		Total =
Requirement fulfillment			Organization of the application		
Validation			Representation and Integration of Database		
Verification			Graphical User Interface		

Topics
Introduction
Case Study
ER Diagram
Normalization & Finalization
Schema Diagram
Database Query
Transition
Features and Facilities
Conclusion

Introduction:

PropertyRentalServices is a desktop-based property rental management application developed using C# Windows Forms and Microsoft SQL Server. The system is designed to digitize and streamline the entire lifecycle of a rental property — from listing and browsing to booking, payment, and review — through a structured multi-role platform.

The application supports three distinct user roles: SuperAdmin, Owner, and Customer. Each role operates within a clearly defined scope of permissions and responsibilities, ensuring data integrity and a tailored user experience. Owners can list, manage, and promote their properties; customers can search, filter, book, pay, and review; and the SuperAdmin oversees all system operations, monitors revenue, and manages platform quality.

The underlying database — PropertyRentalDB — is built on SQL Server and follows a fully normalized relational schema (up to 3NF/BCNF). It consists of six core tables: Users, Property, Booking, Payment, Review, and Offer. The system integrates real-world database concepts including foreign key constraints, views, triggers, stored procedures, and parameterized queries to ensure security, performance, and maintainability.

This report documents the complete database design process — including entity-relationship modeling, normalization analysis, SQL query documentation, and system architecture — providing a comprehensive reference for both the academic submission and future development.

Case Study:

John is a property owner in Dhaka who has three rental apartments — one in Gulshan, one in Dhanmondi, and one near Mirpur. He registers on PropertyRentalServices as an Owner and lists all three properties with details such as location, price per night, number of bedrooms, and a description. He also applies a seasonal discount offer of 10% on his Gulshan property for the month of June.

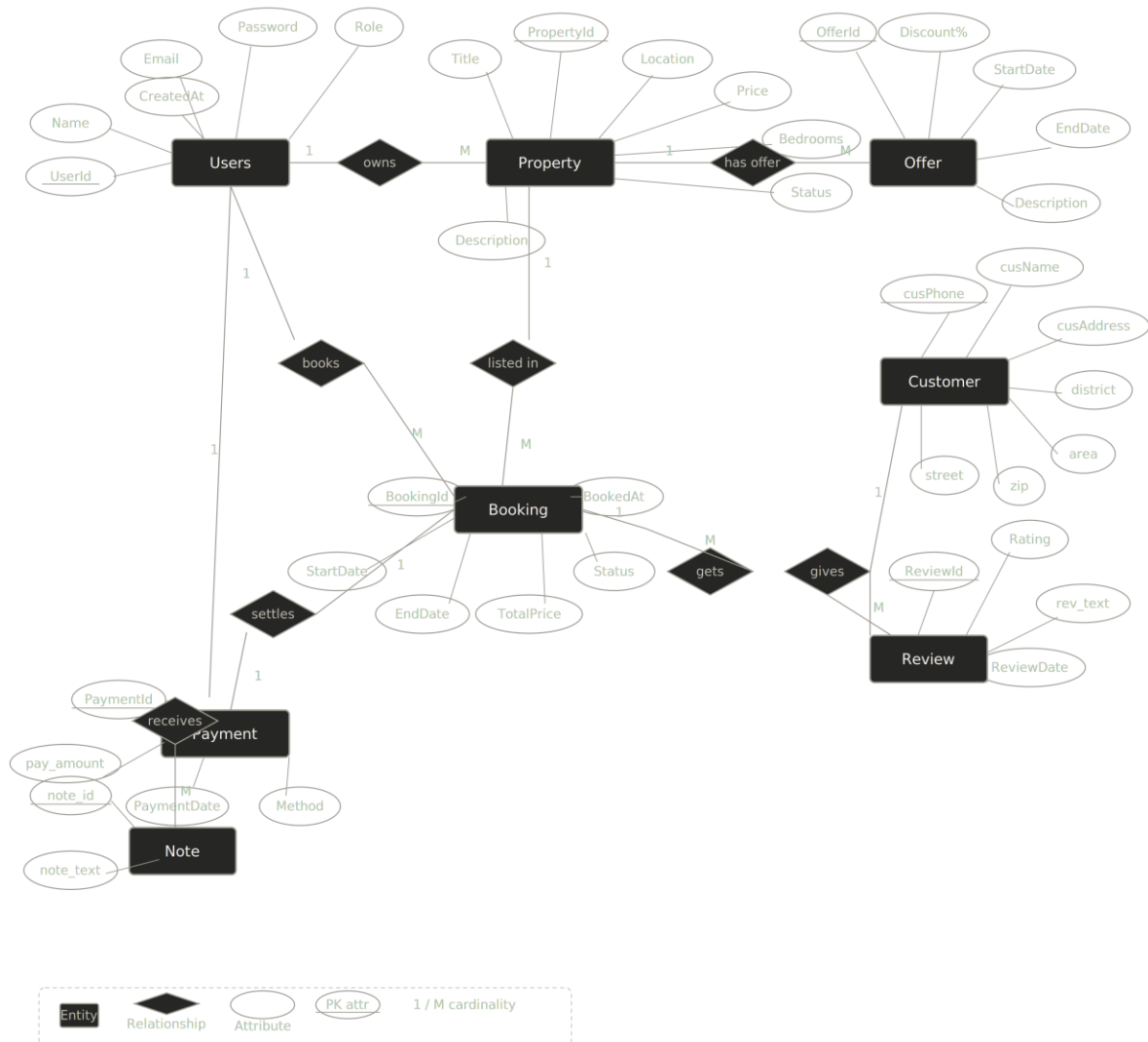
Alice is a working professional looking for a short-term rental in Gulshan. She registers as a Customer, logs in, and uses the Browse Properties feature to filter by location (Gulshan), price range (3,000–7,000 BDT), and number of bedrooms (2). The system displays John's Gulshan apartment with its average rating, active offer, and review count.

Alice selects the property, checks the available dates, and proceeds to the booking form. She selects her check-in and check-out dates, and the system automatically calculates the total price — applying the 10% discount using the active offer. She confirms the booking and completes the payment via the Payment Form, selecting her preferred payment method. The system records the booking and payment, and automatically updates the property status to Booked.

After her stay, Alice submits a review with a rating of 5 stars and a comment. The review is visible to future customers browsing the property. John can view his earnings and booking history from his Owner Dashboard, while the SuperAdmin can monitor all transactions, generate revenue reports (including the platform's 10% commission share), and remove underperforming owners based on average rating thresholds.

This scenario demonstrates how PropertyRentalServices creates a closed-loop rental ecosystem — connecting property owners and customers through a transparent, secure, and efficient platform.

ER Diagram:



Database Normalization

Owns: one to many relationship.

UNF: store_name, store id, bankName, branch_name, accNo, district, area, zip, street, OwnerId, Title, Location, Price, Bedrooms, Status, Description, CreatedAt.

1NF: store_name, store id, bankName, branch_name, accNo, district, area, zip, street, OwnerId, Title, Location, Price, Bedrooms, Status, Description, CreatedAt.

2NF:

1. store_name, store id, bankName, branch_name, accNo, district, area, zip, street.

2. PropertyId, OwnerId, Title, Location, Price, Bedrooms, Status, Description, CreatedAt, store id_(FK)

3NF: Same as 2NF.

Creates: one to many relationship.

UNF: store_name, store id, bankName, branch_name, accNo, district, area, zip, street, d id, status, weight, productType, collectAmount.

1NF: store_name, store id, bankName, branch_name, accNo, district, area, zip, street, d id, status, weight, productType, collectAmount.

2NF:

1. store_name, store id, bankName, branch_name, accNo, district, area, zip, street.

2. d id, status, weight, productType, collectAmount, store id_(FK).

3NF: Same as 2NF.

Books: one to many relationship.

UNF: UserId, Name, Email, Password, Role, CreatedAt, BookingId, PropertyId, StartDate, EndDate, TotalPrice, Status, BookedAt.

1NF: UserId, Name, Email, Password, Role, CreatedAt, BookingId, PropertyId, StartDate, EndDate, TotalPrice, Status, BookedAt.

2NF:

1. UserId, Name, Email, Password, Role, CreatedAt.

2. BookingId, PropertyId, StartDate, EndDate, TotalPrice, Status, BookedAt, UserId_(FK).

3NF: Same as 2NF.

Listed in: one to many relationship.

UNF: PropertyId, OwnerId, Title, Location, Price, Bedrooms, Status, Description, BookingId, CustomerId, StartDate, EndDate, TotalPrice, BookedAt.

1NF: PropertyId, OwnerId, Title, Location, Price, Bedrooms, Status, Description, BookingId, CustomerId, StartDate, EndDate, TotalPrice, BookedAt.

2NF:

1. PropertyId, OwnerId, Title, Location, Price, Bedrooms, Status, Description.

2. **BookingId**, CustomerId, StartDate, EndDate, TotalPrice, Status, BookedAt, **PropertyId**_(FK).

3NF: Same as 2NF.

Settles: one to one relationship.

UNF: **BookingId**, PropertyId, CustomerId, StartDate, EndDate, TotalPrice, Status, BookedAt, **PaymentId**, Amount, PaymentDate, Method.

1NF: **BookingId**, PropertyId, CustomerId, StartDate, EndDate, TotalPrice, Status, BookedAt, **PaymentId**, Amount, PaymentDate, Method.

2NF:

1. **BookingId**, PropertyId, CustomerId, StartDate, EndDate, TotalPrice, Status, BookedAt.

2. **PaymentId**, Amount, PaymentDate, Method, **BookingId**_(FK).

3NF: Same as 2NF.

Receives: one to many relationship.

UNF: **PropertyId**, OwnerId, Title, Location, Price, Bedrooms, Status, **ReviewId**, UserId, Rating, Comment, ReviewDate.

1NF: **PropertyId**, OwnerId, Title, Location, Price, Bedrooms, Status, **ReviewId**, UserId, Rating, Comment, ReviewDate.

2NF:

1. **PropertyId**, OwnerId, Title, Location, Price, Bedrooms, Status, Description.

2. **ReviewId**, UserId, Rating, Comment, ReviewDate, **PropertyId**_(FK).

3NF: Same as 2NF.

Writes: one to many relationship.

UNF: **UserId**, Name, Email, Role, **ReviewId**, PropertyId, Rating, Comment, ReviewDate.

1NF: **UserId**, Name, Email, Role, **ReviewId**, PropertyId, Rating, Comment, ReviewDate.

2NF:

1. **UserId**, Name, Email, Password, Role, CreatedAt.

2. **ReviewId**, PropertyId, Rating, Comment, ReviewDate, **UserId**_(FK).

3NF: Same as 2NF.

Has offer: one to many relationship.

UNF: **PropertyId**, Title, Location, Price, Status, **OfferId**, DiscountPercent, StartDate, EndDate, Description.

1NF: **PropertyId**, Title, Location, Price, Status, **OfferId**, DiscountPercent, StartDate, EndDate, Description.

2NF:

1. **PropertyId**, OwnerId, Title, Location, Price, Bedrooms, Status, Description.

2. **OfferId**, DiscountPercent, StartDate, EndDate, Description, **PropertyId**_(FK).

3NF: Same as 2NF.

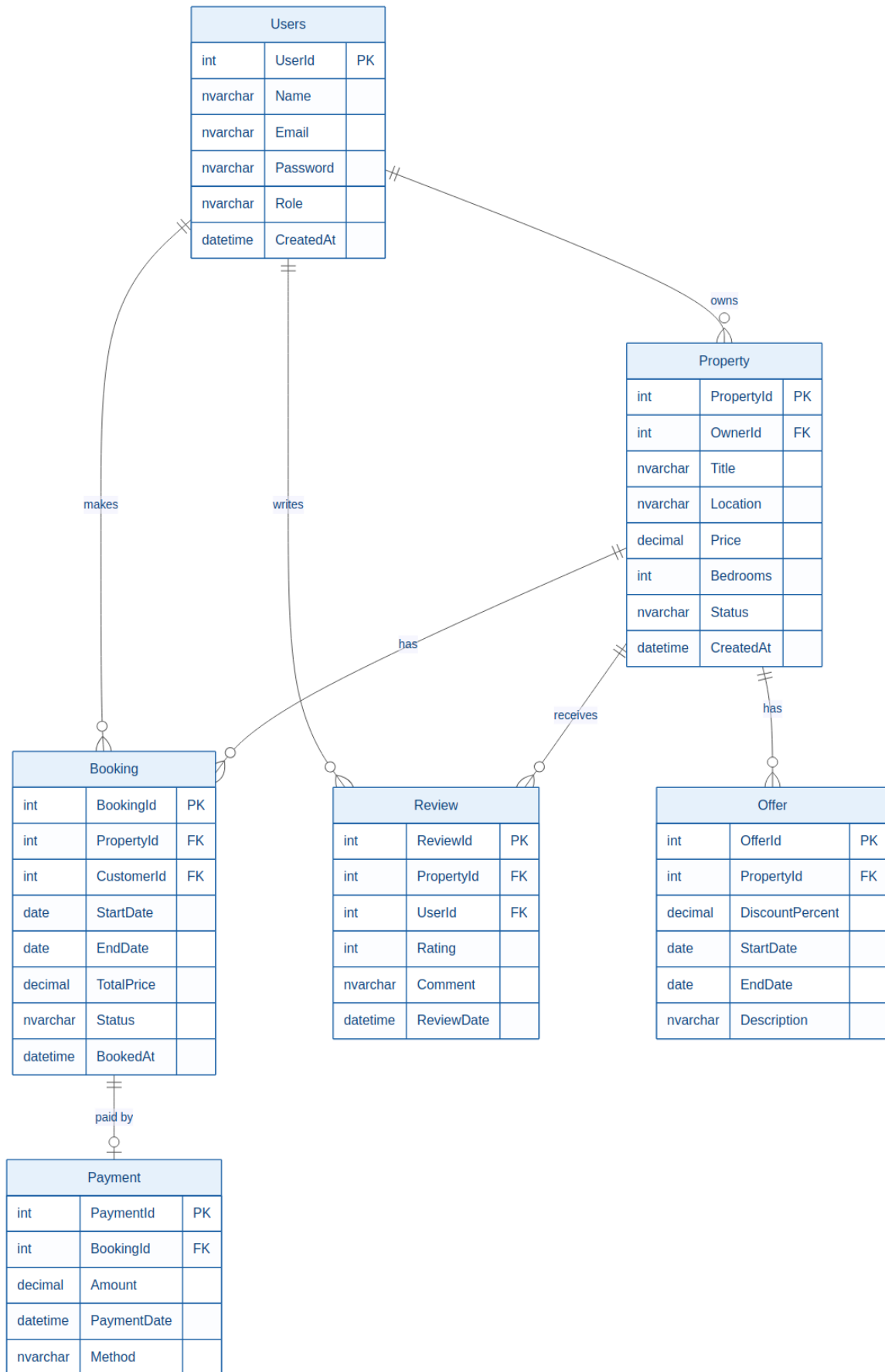
Finalization

1. **Users:** Name, Email, Password, Role, CreatedAt, **UserId**.
2. **Property:** OwnerId, Title, Location, Price, Bedrooms, Status, Description, CreatedAt, **PropertyId**, OwnerId_(FK).
3. **Booking:** PropertyId, CustomerId, StartDate, EndDate, TotalPrice, Status, BookedAt, **BookingId**, PropertyId_(FK), CustomerId_(FK).
4. **Payment:** Amount, PaymentDate, Method, **PaymentId**, BookingId_(FK).
5. **Review:** PropertyId, UserId, Rating, Comment, ReviewDate, **ReviewId**, PropertyId_(FK), UserId_(FK).
6. **Offer:** DiscountPercent, StartDate, EndDate, Description, **OfferId**, PropertyId_(FK).

Final Normalized Tables (3NF):

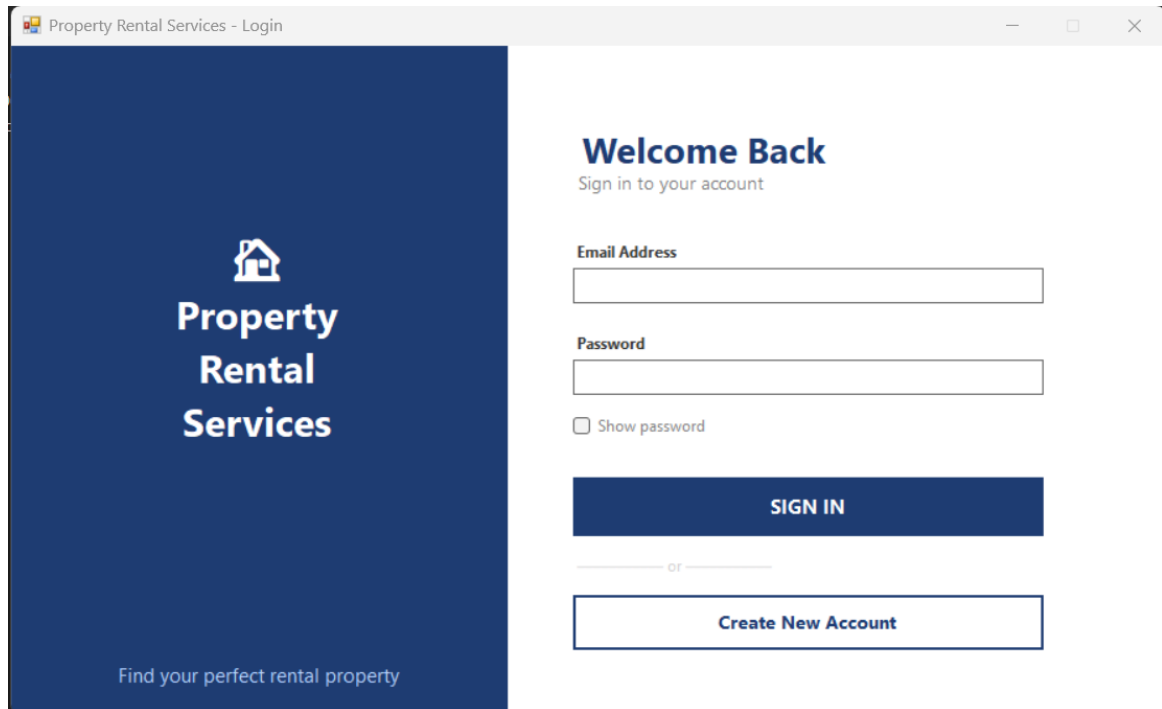
1. **Users:** **UserId**, Name, Email, Password, Role, CreatedAt.
2. **Property:** **PropertyId**, OwnerId_(FK), Title, Location, Price, Bedrooms, Status, Description, CreatedAt.
3. **Booking:** **BookingId**, PropertyId_(FK), CustomerId_(FK), StartDate, EndDate, TotalPrice, Status, BookedAt.
4. **Payment:** **PaymentId**, BookingId_(FK), Amount, PaymentDate, Method.
5. **Review:** **ReviewId**, PropertyId_(FK), UserId_(FK), Rating, Comment, ReviewDate.
6. **Offer:** **OfferId**, PropertyId_(FK), DiscountPercent, StartDate, EndDate, Description.

Schema Diagram:



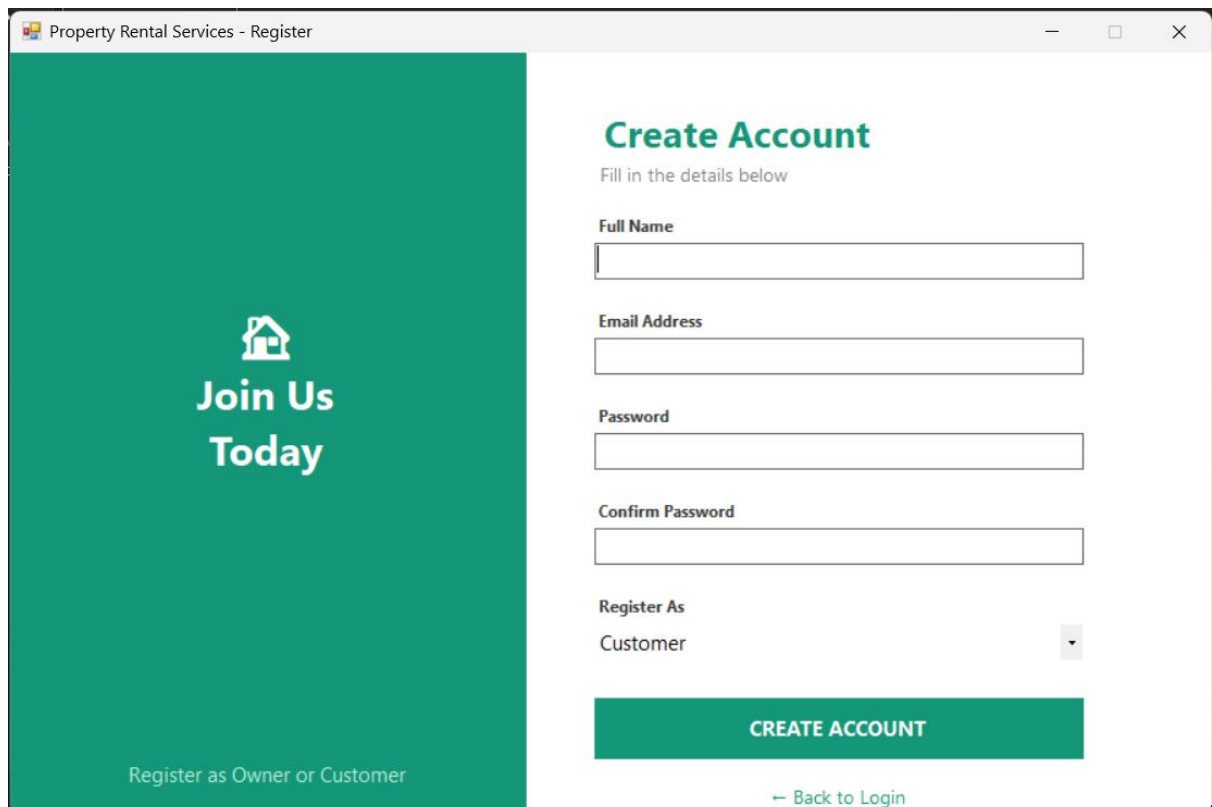
Transition:

Login Page:



The screenshot shows a web browser window titled "Property Rental Services - Login". The page is split into two main sections. On the left, a dark blue vertical banner contains a white house icon, the text "Property Rental Services" in a large, bold, white font, and the tagline "Find your perfect rental property" in a smaller white font at the bottom. On the right, the main content area has a white background. It starts with the heading "Welcome Back" in a bold blue font, followed by the subtext "Sign in to your account" in a smaller blue font. Below this are two input fields: "Email Address" and "Password", both with white borders and no text inside. Under the password field is a checkbox labeled "Show password". Below the inputs is a large, solid dark blue button with the text "SIGN IN" in white, uppercase letters. Below this button is a horizontal line with the word "or" in the center. Underneath the line is a white button with a dark blue border and the text "Create New Account" in dark blue, uppercase letters.

Create Account:



The screenshot shows a web browser window titled "Property Rental Services - Register". The page is split into two main sections. On the left, a green vertical banner contains a white house icon, the text "Join Us Today" in a large, bold, white font, and the tagline "Register as Owner or Customer" in a smaller white font at the bottom. On the right, the main content area has a white background. It starts with the heading "Create Account" in a bold green font, followed by the subtext "Fill in the details below" in a smaller green font. Below this are four input fields: "Full Name", "Email Address", "Password", and "Confirm Password", all with white borders and no text inside. Below the "Password" field is a dropdown menu labeled "Register As" with "Customer" selected and a small downward arrow icon. Below the dropdown is a large, solid green button with the text "CREATE ACCOUNT" in white, uppercase letters. At the bottom right of the page is a link that says "← Back to Login" in a small green font.

Super Admin Panel:

SuperAdmin Dashboard - Property Rental Services

SuperAdmin Panel | Welcome, Super Admin

Logout

Total Users

5

Properties

5

Bookings

1

Revenue (₳)

4,500

All Users

All Properties

All Bookings

All Reviews

Revenue (10%)

Refresh

Delete Owner (Low Rating)

Userid	Name	Email	Role	CreatedAt
5	raonak	raonak@gmail.com	Owner	4/28/2026 9:24 AM
4	Ankon	ankon4200@gmail.com	Customer	4/28/2026 8:41 AM
2	John Owner	john@owner.com	Owner	4/24/2026 3:00 AM
3	Alice Customer	alice@customer.com	Customer	4/24/2026 3:00 AM
1	Super Admin	admin@rental.com	SuperAdmin	4/24/2026 3:00 AM

Owner Dasboard:

Owner Dashboard - Property Rental Services

Owner Dashboard | Welcome, John Owner

Logout

My Properties

4

Confirmed Bookings

1

Total Revenue (₳)

4,500

Revenue after 10% (₳)

4,050

Manage Properties

Earnings

My Offers

Property Title

Cozy Downtown Apartment

Location

Dhaka, Gulshan

Price per Night (₳)

5000.00

Bedrooms

2

Status

Booked

Description

Modern apartment in the heart of Gulshan

+ Add

Update

Delete

PropertyId	Title	Location	Price	Bedrooms	Status	Description	CreatedAt
1	Cozy Downt...	Dhaka, Guls...	5000.00	2	Booked	Modern ap...	4/24/2026 ...
2	Spacious Fa...	Dhaka, Dha...	8500.00	4	Available	Large famil...	4/24/2026 ...
3	Studio Nea...	Dhaka, Mir...	2500.00	1	Available	Perfect for ...	4/24/2026 ...
4	Luxury Pent...	Dhaka, Ban...	15000.00	3	Available	Top floor lu...	4/24/2026 ...

Customer Panel:

Customer Dashboard - Property Rental Services

Property Rental | Welcome, Alice Customer

Logout

Browse Properties

Booking Cart

My Bookings

My Reviews

Search:

Location:

Bedrooms:

Status:

Min Price (b):

Max Price (b):

Search

Reset

Add to Cart

View Reviews

PropertyId	Title	Location	Price/Night	Bedrooms	Status	Avg Rating	Reviews	Active Offer
3	Studio Near Univer...	Dhaka, Mirpur	2500.00	1	Available	0	0	15.00%
1	Cozy Downtown A...	Dhaka, Gulshan	5000.00	2	Booked	0	0	10.00%
2	Spacious Family H...	Dhaka, Dhanmondi	8500.00	4	Available	0	0	No Offer
4	Luxury Penthouse	Dhaka, Banani	15000.00	3	Available	0	0	No Offer

Features and Facilities:

PropertyRentalServices delivers the following key features:

1. Multi-Role Authentication — Separate login flows for SuperAdmin, Owner, and Customer with role-based access control.
2. Property Management — Owners can add, edit, and delete property listings with full attribute control.
3. Advanced Property Search & Filter — Customers can filter by keyword, location, price range, bedroom count, and availability status.
4. Offer & Discount System — Owners can define time-limited percentage discounts; discounts are automatically applied during booking price calculation.
5. Booking & Payment Processing — Customers can book properties for specific date ranges; payment is recorded with method and linked to the booking.
6. Review & Rating System — Customers can rate and comment on properties they have experienced; duplicate reviews are prevented.
7. Admin Revenue Reporting — The SuperAdmin can view a detailed revenue report showing total rent collected and the platform's 10% commission per confirmed booking.
8. Owner Performance Monitoring — The SuperAdmin can identify and remove owners whose average property rating falls below a defined threshold.
9. Historical Price Preservation — TotalPrice in Booking is stored at booking time, ensuring historical accuracy even if property prices are later updated.
10. Data Integrity via Constraints — All tables enforce NOT NULL, UNIQUE, CHECK, and FOREIGN KEY constraints to maintain clean and consistent data.

Conclusion:

PropertyRentalServices successfully demonstrates the application of database management principles in a real-world desktop application context. The project covers the complete lifecycle of database design: from entity-relationship modeling and normalization analysis through to SQL query implementation and C# application integration.

The PropertyRentalDB schema achieves full 3NF compliance across all six tables — Users, Property, Booking, Payment, Review, and Offer — with a single justified denormalization (Booking.TotalPrice) for historical accuracy. The schema enforces referential integrity through foreign keys and ensures data validity through CHECK constraints.

The system's three-role architecture (SuperAdmin, Owner, Customer) reflects real-world business logic, and the SQL query library — covering DDL, DML, views, aggregations, joins, and parameterized application queries — provides a solid, secure foundation for the C# Windows Forms front end.

Overall, PropertyRentalServices demonstrates how a well-designed relational database, combined with thoughtful application architecture, can deliver a reliable, scalable, and user-friendly property rental platform.